

MixBytes()

Lido AAVE Migrator Security Audit Report

JUNE 08, 2026

Table of Contents

1. Introduction	2
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	4
1.4 Security Audit Methodology	7
Stage 1: Interim Audit	7
Stage 2: Re-audit & Mainnet Deployment Verification	8
1.5 Risk Classification	10
1.6 Summary of Findings	11
2. Findings Report	12
2.1 Critical	12
2.2 High	12
2.3 Medium	12
2.4 Low	12
3. About MixBytes	13

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

`AaveMigrator` is a one-shot, operator-driven utility that moves a leveraged Aave V3 position (one collateral + one variable-rate debt) from a source Mellow `Subvault` into a target Mellow `Subvault`. It is single-purpose, non-upgradeable, `Ownable`, protected by `ReentrancyGuard`, and never custodies funds itself. All asset movement is driven by the two subvaults via their `ICallModule.call(...)` interface, which routes each call through the subvault's `Verifier`. A single `migrate(maxUtilizationD18, percentageD6)` call runs up to 30 iterations of `target.borrow(debt) -> target.transfer(debt to source) -> source.repay(debt) -> source.withdraw(collateral to target) -> target.supply(collateral)`, optionally migrating only a fraction of the source debt per call; the contract remains reusable for further partial calls until the source debt is fully retired, at which point the `migrated` flag latches.

The codebase was reviewed via manual line-by-line analysis on commit `8016581cec37aff1892c942df37b4ae8c073ad72`, followed by a re-audit pass on commit `82b1849c30f26f0c2a7940feb28bc6c5937ca791` (PR-115) and a mainnet-deployment verification pass against the three Plasma deployments listed below. The review cross-references the prior audits of the sibling contracts `GGVMigrator` (commit `da932764a3e9fbc71127f39aa1006a2ddf73d46c`) and `GGVMigratorV2` (commit `07adcc04ecc3279edb9de0b67194ab0a91dbe025`). `AaveMigrator` is a generalisation of `GGVMigratorV2`: instead of hardcoded GGV / WETH / weETH addresses, it takes the source subvault, target subvault, debt asset and collateral asset as constructor parameters, and additionally accepts numeric thresholds (`minAllowedSourceHFD18`, `minAllowedTargetHFD18`, `minAllowedMigratingDebt`, `maxAllowedError`) at deployment time. The loop body, partial-migration semantics, equity-drift check, debt-band check, and `kill()` switch are direct ports from `GGVMigratorV2`. The interim review was conducted over 1 working day via manual code review and a proprietary AI-assisted analysis tool.

The diff against `GGVMigratorV2` is narrow and additive at the architecture level. No new Critical, High, Medium, or Low issues were identified for `AaveMigrator`. Between the initial review commit and the deployed bytecode, the client landed a follow-up fix in commit `82b1849c30f26f0c2a7940feb28bc6c5937ca791` (PR-115) that adds token-decimals scaling to `calculateSteps` so the migration works correctly for non-18-decimal assets (in particular the 6-decimal `USDT0` debt asset used in the Plasma deployments); we reviewed that change and confirmed it behaves as intended. The two Low-severity items raised against the V1/V2 lineage carry over by design and are not re-reported: namely the equity-drift bound being enforced per call rather than across partial migrations (`GGVMigratorV2` Low #1, Acknowledged), and `kill()` being callable at any time including after a `PartiallyMigrated` outcome (`GGVMigrator` V1 Low #2, Acknowledged).

Both behaviours are preserved verbatim in [AaveMigrator](#); the previously-acknowledged status applies.

Constructor parameters of the three Plasma deployments were cross-checked against on-chain canonical addresses (Aave V3 Pool, Aave V3 oracle, collateral / debt reserves, owning Safe) and against the corresponding [subvaultAt\(...\)](#) slots of the strETH and earnUSDe vaults published by Mellow.

Out of scope:

- Aave V3 protocol (pool, oracle, aToken / variableDebtToken implementations).
- The Mellow flexible-vaults framework, including [Subvault](#), [CallModule](#), [Verifier](#), and [VerifierModule](#).
- The Aave oracle / Chainlink feeds.
- Any off-chain components (operator tooling, deployment scripts, keepers).

Overall, [AaveMigrator](#) is a faithful generalisation of the previously-audited [GGVMigratorV2](#): atomic, defensively bounded by HF floors and equity / debt invariants, with no new findings identified during this review.

1.3 Project Overview

Summary

Title	Description
Client	Lido
Category	Liquid Staking
Project	AAVE Migrator
Type	Solidity
Platform	EVM
Timeline	03.06.2026 - 08.06.2026

Scope of Audit

File	Link
src/utils/AaveMigrator.sol	src/utils/AaveMigrator.sol

Versions Log

Date	Commit Hash	Note
03.06.2026	8016581cec37aff1892c942df37b4ae8c073ad72	Initial Commit
08.06.2026	82b1849c30f26f0c2a7940feb28bc6c5937ca791	Commit with Updates

Mainnet Deployments

File	Address	Blockchain
AaveMigrator.sol	0x00000000B984aFe3a14618768662829a78928bbe	Plasma
AaveMigrator.sol	0x00000000e8887A95a9bE39de46143c8E58794bE4	Plasma
AaveMigrator.sol	0x00000000C7Bf3227130841e959FD73EEDDB16537	Plasma

The same [AaveMigrator](#) bytecode is deployed three times on Plasma – once per (collateral, debt) pair – to move the corresponding leveraged Aave V3 positions from [strETH](#) subvaults into [earnUSDe](#) subvaults: [0x00000000B984aFe3a14618768662829a78928bbe](#) handles USDe / USDT0 (strETH subvault0 -> earnUSDe subvault0), [0x00000000e8887A95a9bE39de46143c8E58794bE4](#) handles sUSDe / USDT0 (strETH subvault0 -> earnUSDe subvault0), and [0x00000000C7Bf3227130841e959FD73EEDDB16537](#) handles syrupUSDT / USDT0 (strETH subvault1 -> earnUSDe subvault1).

All three deployments were reviewed and their on-chain state matches the audited scope. Runtime bytecode at each address was reproduced byte-for-byte by compiling commit [82b1849c30f26f0c2a7940feb28bc6c5937ca791](#) with the repository's [foundry.toml](#) settings (solc 0.8.25).

The shared on-chain configuration across all three deployments matches the Plasma deployment script ([scripts/plasma/MigrationAave.s.sol](#)) and Mellow's published deployment docs:

- `owner()` = [0x5Dbf9287787A5825beCb0321A276C9c92d570a75](#) (the curator Safe, see below).
- `pool()` = [0x925a2A7214Ed92428B5b1B090F80b25700095e12](#) (Aave V3 Pool on Plasma, `POOL_REVISION()` == 11, `ADDRESSES_PROVIDER()` == [0x061D8e131F26512348ee5FA42e2DF1bA9d6505E9](#)).
- `oracle()` = [0x33E0b3fc976DC9C516926BA48Cf0A9E10a2aAA5](#) (the Aave V3 oracle wired into the same `PoolAddressesProvider`).
- `debt()` = [0xB8CE59FC3717ada4C02eaDF9682A9e934F625ebb](#) (USDT0, 6 decimals); `debtUnit()` == 1e6.
- `debtAToken()` = [0x5D72a9d9A9510Cd8cBdBA12aC62593A58930a948](#) and `debtVariableDebtToken()` = [0xCBBC427b5658672768E11BFDA00879839DB4785F](#) – both match `pool.getReserveAToken(USDT0)` and `pool.getReserveVariableDebtToken(USDT0)` queried directly against the Aave V3 Pool.
- `minAllowedSourceHFD18()` = `minAllowedTargetHFD18()` = [1.01e18](#) (matches the 1.01 ether in the deploy script and the safety margin used in V1 / V2).
- `minAllowedMigratingDebt()` = [1e10](#) (the smallest per-call debt the contract will accept; for the 6-decimal USDT0 debt this is \$10 000 worth, well above the per-call drift tolerance).
- `maxAllowedError()` = [1e6](#) – equity-drift / residual-debt tolerance, interpreted in the Aave oracle's 8-decimal base currency, i.e. ~\$0.01 per call.

For each deployment the migrator's `collateralAToken()` matches `pool.getReserveAToken(collateral)` queried directly against the Aave V3 Pool, confirming that the constructor's `pool.getReserveAToken(...)` / `pool.getReserveVariableDebtToken(...)` calls resolved to the canonical Aave V3 reserve tokens for those assets. The source / target subvaults match the corresponding `subvaultAt(...)` slots of the strETH ([0x841e213864046111E43d237703d71FaBe91Ef9e0](#)) and earnUSDe ([0x49DAb986A4288bE616f44733d56397d3410fD331](#)) vaults – `subvaultAt(0)` for the USDe and sUSDe deployments, `subvaultAt(1)` for the syrupUSDT deployment – published in [Mellow's strETH deployment documentation](#) and [the earnUSDe deployment documentation](#).

The owning Safe at `0x5Dbf9287787A5825beCb0321A276C9c92d570a75` uses the canonical Gnosis Safe v1.4.1 singleton (`0x29fcb43b46531bca003ddc8fcb67ffe91900c762`, `VERSION() == "1.4.1"`), `getThreshold() == 3`, 8 owners (all EOAs at the time of review), `nonce() == 69` (active operational use), no Guard contract installed, and no modules enabled (the `SENTINEL_MODULES` traversal returns an empty array). Compromise of the owning Safe is therefore bounded by the 3-of-8 threshold across independent EOA signers under canonical Safe code, with no bypass paths.

1.4 Security Audit Methodology

Stage 1: Interim Audit

Project Architecture Review

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM, HOW THEY INTERACT ARCHITECTURALLY, AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

Activities include:

- understanding overall system design and contract interactions
- mapping responsibilities of each component and protocol workflows
- conducting a thorough line-by-line review of contracts and modules
- tracing execution paths and mapping state transitions
- identifying trust boundaries and external integrations
- forming an independent architectural understanding directly from code before validating documentation
- running the project test suite to observe implementation behavior and encoded invariants
- reviewing documentation, specifications, READMEs, and deployment guides and reconciling them with code

Adversarial Code Review

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

Approach includes:

- analyzing the codebase from an attacker's perspective, focusing on what can fail rather than only confirming intended behavior
- searching for overlooked edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- attempting to violate protocol invariants and identifying scenarios where system guarantees break
- analyzing economic attack surfaces including liquidation logic, oracle dependencies, vault accounting, and cross-contract interactions
- writing targeted tests, modelling adversarial scenarios, applying mutation testing and fuzzing, and developing proof-of-concept exploits
- conducting independent exploit path discovery by each auditor to reduce anchoring bias
- reviewing high-risk code collaboratively in pair-auditing sessions led by the Team Lead

Systematic Vulnerability Analysis

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

Activities include:

- reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- analyzing common DeFi attack classes such as reentrancy, accounting manipulation, oracle manipulation, privilege escalation, and state desynchronization
- using proprietary AI tooling to surface candidate issues across broader attack vectors
- manually validating and triaging AI-generated candidates

Consolidation of Auditors' Reports

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

Activities include:

- cross-checking findings across auditors
- consolidating and deduplicating issues
- resolving discrepancies in severity assessments
- using proprietary AI tooling to assist with report drafting and consistency checks
- manually reviewing and finalizing the report narrative
- preparing the interim audit report for client review

Stage 2: Re-audit & Mainnet Deployment Verification

Re-audit

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THAT THE UPDATED CODE DOES NOT INTRODUCE NEW VULNERABILITIES.

Activities include:

- confirming each remediation matches the original recommendation
- documenting rationale for any unresolved findings
- verifying modified logic and integration points remain secure
- executing the project's tests after remediation, including targeted coverage of fixed areas
- using proprietary AI tooling on the updated codebase to surface potential new issues

Mainnet Deployment Verification

OBJECTIVE: PERFORM FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC AUDIT REPORT.

Activities include:

- verifying deployed contract bytecode against audited source code across networks
- confirming deployed bytecode matches the build produced from the audited commit using identical compiler settings
- reviewing constructor and initializer arguments used in deployment
- verifying proxy deployment order, initialization flow, and admin configuration
- ensuring implementations are not left uninitialized or exposed to initialization front-running
- publishing the final report after alignment between client and audit team

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.
- **Medium** – Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.
- **Low** – One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

Likelihood

- **High** – An event with an estimated 50-60% probability of occurring within a year, which can be triggered by any actor (e.g., due to a market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the contract owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed in order to minimize potential risks.
- **Medium** – Recommended to be fixed to improve security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and efficiency.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and the issue no longer impacts the security of the protocol.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but the issue has not been fully resolved.
- **Acknowledged** – The recommended fixes have not been implemented, and the finding remains unresolved or has been accepted by the project team without code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	0
Low	0

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

Not Found

2.4 Low

Not Found

3. About MixBytes

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems.

The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle – from early architecture design to production audits and ongoing protocol evolution.

Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including **Lido**, **Aave**, **Curve**, **1inch**, **OKX**, **Mantle**, **Fluid**, **Gearbox**, **Resolv**, and others, helping teams identify vulnerabilities, strengthen protocol design, and improve the robustness of decentralized systems.

To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review. These tools augment the work of senior auditors but do not replace expert analysis.

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

- **Design Review.** Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.
- **AI Tooling.** AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.
- **Smart Contract Audit.** Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.
- **Security Retainer.** Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

Contacts



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



hello@mixbytes.io